Hacia un traductor especializado en jergas y dialectos del español: generación de datos sintéticos multi-fuente y fusión de modelos de lenguaje pequeños

Anderson García Mariana Malagón
Paula Lozano

Ingeniería de Sistemas, Escuela Colombiana de Ingeniería Julio Garavito

13 de noviembre de 2026

Resumen

Los sistemas de traducción automática contemporáneos alcanzan un desempeño alto en el registro estándar de un idioma, pero tienden a fallar cuando el texto de entrada contiene jerga o expresiones dialectales, produciendo traducciones literales que distorsionan o invierten el significado original. Esta limitación no es meramente incómoda: existen casos documentados en contextos de salud y de justicia donde un error de esta naturaleza tuvo consecuencias severas para las personas afectadas. Este documento presenta la propuesta inicial de un proyecto que investiga la viabilidad de construir un traductor especializado en jerga y dialectos del español y el inglés mediante el ajuste fino (*fine-tuning*) de un modelo de lenguaje pequeño (*Small Language Model*, SLM) sobre datos sintéticos generados por múltiples modelos de lenguaje de gran escala (LLMs). Se propone, además, investigar si la fusión de pesos (*model merging*) de modelos entrenados con datos provenientes de distintos generadores sintéticos permite consolidar el conocimiento de varias fuentes en un único modelo eficiente, sin incurrir en los costos de un *ensemble*. Estas son, ante todo, preguntas de investigación; que el resultado además sirva como propuesta de producto frente a alternativas como Google Translate o DeepL es una pregunta distinta, que este documento aborda por separado. Este documento delimita el problema, revisa el estado del arte en traducción dialectal, generación de datos sintéticos y fusión de modelos, identifica los vacíos que motivan el proyecto, describe la propuesta general de solución, y presenta la arquitectura detallada, un prototipo funcional (servicio de traducción vía API, con observabilidad y seguridad básica) y resultados preliminares de calidad y latencia para un solo generador sintético. La comparación completa entre los tres generadores considerados y la fusión de modelos se desarrollarán en la fase final del proyecto.

Palabras clave: traducción automática, dialectos, jerga, modelos de lenguaje pequeños, datos sintéticos, fusión de modelos, destilación de conocimiento.

1. Introducción

1.1. Contexto

Hoy en día entrenar modelos de lenguaje ya no depende de un solo generador de datos: es común mezclar salidas de varios LLMs distintos y luego fusionar los modelos resultantes, en lugar de apostarle a uno solo. Esa combinación (síntesis de varias fuentes más fusión) es relativamente nueva y todavía no se ha probado a fondo para tareas específicas del español, como traducir jerga o dialectos regionales.

Y ahí surge una pregunta que cualquier hispanohablante se ha hecho sin necesidad de ser investigador: si hoy entrenamos IA con técnicas tan sofisticadas para innumerables propósitos, ¿por qué seguimos sin lograr que entienda que "dar papaya.º "qué galleta"significan cosas completamente distintas según de dónde venga quien las dice?

Este no es un problema nuevo ni menor. En 1980, un joven cubanoamericano de Florida, Willie Ramírez, quedó cuadripléjico porque el personal médico tradujo 'intoxicado' como *intoxicated* en vez de *poisoned*, y lo trataron por sobredosis cuando en realidad tenía una hemorragia cerebral. El hospital

terminó pagando una indemnización de 71 millones de dólares. Han transcurrido más de cuarenta años y, aunque los modelos actuales traducen el español estándar con un nivel de precisión considerablemente alto, un estudio de ACL 2025 (Mi, Villavicencio y Moosavi) muestra que los LLMs más recientes siguen fallando específicamente al interpretar el contexto de expresiones idiomáticas. Es decir, el problema no radica en que la IA traduzca de forma deficiente, sino en que, pese a un desempeño notable en términos generales, persiste sin resolverse este aspecto particular.

Por eso, este trabajo no busca solo construir una herramienta más de traducción, sino generar evidencia sobre si combinar generadores sintéticos con fusión de modelos realmente funciona para esta tarea concreta, algo que hasta ahora nadie ha probado de forma sistemática para el español.

1.2. Nicho identificado

Antes de entrar en el estado del arte completo, vale la pena situar con precisión el vacío que motiva este proyecto. Lo ubicamos en la intersección de tres líneas de trabajo que, hasta donde alcanzamos a revisar (ACL Anthology, Semantic Scholar, arXiv, Google Scholar, y también literatura gris como blogs técnicos y reportes de industria), no hemos encontrado combinadas en ningún lado:

(I) **Traducción (no solo reconocimiento) de jerga y dialectos del español mediante *fine-tuning*.** Para otros idiomas ya hay resultados concretos. Yakhni y Chehab ajustaron el modelo Aya23-8B para traducir dialecto libanés, generando parte de sus datos de entrenamiento a partir de un libro de gramática [8]. Jerpelea et al. mostraron que un corpus paralelo de cerca de 79 mil ejemplos ya es suficiente para ajustar modelos preentrenados al aromanian [9], y algo parecido se hizo para el šariš, un dialecto eslovaco de bajo recurso [10]. Incluso hay un banco de pruebas de traducción de jerga inglés-chino, con más de 40 mil pares, donde el modelo especializado supera a las líneas base sin ajuste fino [11]. Para el español no encontramos nada parecido: lo que hay se queda en *reconocer* o clasificar la variante dialectal [12, 13], pero no llega a traducirla.

(II) **Comparación controlada de múltiples LLMs como generadores de datos sintéticos, aplicada a una tarea de traducción.** La elección del generador sintético sí se ha tratado como variable experimental en otros contextos: para clasificación de falacias lógicas, con diferencias de hasta 14 puntos porcentuales de F1 según el LLM usado [16], y para diálogo médico en árabe, donde GPT-4o superó consistentemente a Gemini 2.5 Pro en calidad y tasa de alucinación [17]. También hay evidencia de que la diversidad de fuentes sintéticas por sí sola afecta el comportamiento del modelo resultante [18]. Esa comparación aplicada a una tarea de traducción de dialecto o jerga, sin embargo, no la encontramos en ninguna parte.

(III) **Fusión de modelos entrenados sobre datos sintéticos de distintas fuentes, en una tarea generativa de traducción.** La fusión de pesos (*model merging*) ya es una técnica bastante madura para combinar modelos especializados en distintas tareas o idiomas [19–23], y también sirve para combinar modelos entrenados sobre distintos *datasets* en clasificación [25]. En traducción multilingüe la evidencia es más de advertencia que de garantía: la fusión resulta frágil cuando los modelos difieren en el idioma de destino [24], y el trabajo más reciente sobre fusión guiada por destilación multi-maestro reconoce abiertamente que llevarla a tareas generativas queda pendiente [26].

En resumen, cada una de estas tres líneas está resuelta por su lado y con buena evidencia detrás. Lo que nosotros proponemos es juntarlas para el español: traducir (no solo reconocer) la jerga y el dialecto, comparar de forma controlada los generadores que alimentan esa traducción, y consolidar el resultado fusionando modelos.

1.3. Contribuciones esperadas

Este proyecto espera aportar:

(1) evidencia empírica sobre la viabilidad de un SLM especializado en traducir jerga y dialectos del español y el inglés;

(2) una comparación controlada del efecto que tiene el generador de datos sintéticos (entre varios LLMs) sobre la calidad de la traducción resultante;

(3) evidencia sobre si la fusión de modelos (simple o guiada por destilación) permite consolidar el conocimiento de varias fuentes de datos sintéticos en un único modelo eficiente; y

(4) un *pipeline* de generación, validación y evaluación de datos reutilizable para otras variantes dialectales no cubiertas en este trabajo. La Sección 5 detalla estas contribuciones.

1.4. Organización del documento

El resto de este documento se organiza así: la Sección 2 caracteriza el problema y formula las preguntas de investigación. La Sección 3 presenta el estado del arte en traducción dialectal, generación de datos sintéticos y fusión de modelos. La Sección 4 describe la propuesta general de solución. La Sección 5 enumera las contribuciones esperadas. Las Secciones 6-8 corresponden a la entrega de la fase siguiente del proyecto (arquitectura detallada, evidencia de prototipo y resultados preliminares, respectivamente). La Sección 9 describe el trabajo que queda para la fase final del proyecto, y la Sección 10 concluye el documento.

2. Caracterización del problema

2.1. ¿Cuál es el problema?

Los traductores automáticos y los LLMs de uso general están hechos, sobre todo, para el registro estándar del español y el inglés. Cuando alguien usa una expresión de su dialecto (caribeño, andino, rioplatense, mexicano, centroamericano, peninsular) o jerga particular, el sistema tiende a traducirla literal o a leerla desde el registro que mejor conoce, y termina perdiendo lo que la persona quiso decir. Esto no es una sospecha sin evidencia: un estudio de 2026 evaluó a los LLMs con más de 900 palabras que cambian según el país (por ejemplo, "carro" en gran parte de Latinoamérica frente a "coche" en España) en 21 países hispanohablantes, y encontró que los modelos reconocen bien las variantes de España, México/Centroamérica y el Río de la Plata, pero fallan de forma sistemática con el español chileno, una diferencia que no se explica por la cantidad de datos disponibles en internet para cada país [14]. Es decir, el sesgo hacia ciertos dialectos no es solo un problema de escasez de datos, sino de cómo el modelo termina representando (o ignorando) las variantes menos dominantes. Fallas como esta fueron parte de lo que llevó a más de 30 instituciones latinoamericanas a construir Latam-GPT en 2026, un modelo entrenado desde cero con datos locales para no depender de un español que en los modelos de EE.UU. viene sobre todo de España o de textos traducidos del inglés [15].

Que exista un esfuerzo de esa magnitud confirma que el problema es real, pero también deja ver algo clave: esa solución pasa por entrenar un modelo fundacional completo, algo que exige presupuesto y cómputo que casi ningún equipo fuera de una coalición de universidades y gobiernos tiene. Este proyecto va por otro lado: en lugar de un modelo nuevo desde cero, ajustar uno pequeño ya existente con datos sintéticos y fusión de modelos, específicamente para traducir jerga y dialectos del español. Es una escala mucho más modesta, pero más accesible y rápida de probar, y ataca el problema desde un ángulo que Latam-GPT no cubre: no reemplazar el modelo base, sino especializarlo para una tarea puntual sin el costo de entrenar uno nuevo.

2.2. Por qué las soluciones actuales no cierran esta brecha

Vale la pena ser explícitos sobre algo que cualquier lector va a preguntar de inmediato: ¿no resolvió esto ya Google, con la actualización de Translate basada en Gemini de diciembre de 2025? La respuesta corta es que no, al menos no donde importa para este proyecto. Google anunció que Gemini mejora la traducción de "modismos, expresiones locales o jerga", con "calidad de vanguardia" [27], pero el ejemplo público que usan es un modismo genérico en inglés (*"stealing my thunder"*), no variación dialectal *dentro* del español, y no hay evidencia pública de que se haya evaluado contra jerga regional (chilena,

colombiana, rioplatense). De hecho, dos meses después de ese anuncio, el propio estudio de Kawasaki que citamos arriba [14] sigue documentando que los LLMs fallan de forma sistemática en representar la variación léxica geográfica del español, y un estudio de la misma época encuentra disparidades atribuibles a dialectos específicos en el sistema de subtítulos automáticos de YouTube [29]. Esto tiene una consecuencia que cualquiera que haya visto un video con subtítulos traducidos automáticamente reconoce de inmediato: YouTube primero transcribe el audio y después traduce esa transcripción, así que si la transcripción ya viene mal por un acento o dialecto que el sistema no reconoce bien, la traducción hereda y agrava ese error, produciendo subtítulos que aparecen en pantalla con toda la apariencia de estar bien, pero que no dicen lo que la persona realmente dijo [30]. DeepL es todavía más directo: su propia documentación de 2026 dice, textualmente, que el español "latinoamericano" que produce "no reflejará una variante específica, sino un estilo de español general y más neutro" [28]. No es que lo intenten y les falte precisión: no lo intentan.

Tampoco faltan herramientas de nicho —hay traductores enfocados en un solo dialecto, como uno especializado en jerga dominicana—, pero son productos de entretenimiento sin metodología publicada ni evaluación reproducible, pensados para sonar "como local" en una conversación casual, no para un contexto donde el error cuesta caro. Ninguno de los dos extremos —el gigante generalista y el juguete de nicho— hace lo que describimos en la Sección 3: comparar de forma controlada distintos generadores de datos sintéticos, o fusionar los modelos resultantes.

2.3. Un problema que no se agota en el español

El sesgo hacia ciertas variantes tampoco es exclusivo del español. Colombia es un buen ejemplo de por qué el problema es más amplio de lo que hemos planteado hasta acá: además de sus variantes dialectales del español, el país tiene cerca de 65 lenguas indígenas vivas, de las cuales el Ministerio de Cultura no clasifica ninguna como completamente segura y ubica a más de la mitad en algún grado de riesgo de desaparición [31]. La escasez de datos digitales para esas lenguas es todavía más severa que para el dialecto o la jerga del español. Es un problema de otra naturaleza —no es cuestión de matiz o registro, sino de escasez casi total de datos de entrenamiento—, así que no lo incorporamos al alcance técnico de este proyecto. Pero si el *pipeline* que proponemos (semillas curadas, generación sintética, ajuste de un modelo pequeño, evaluación con hablantes nativos) demuestra ser viable para el español, es una base metodológica razonable para explorar, como trabajo futuro, lenguas indígenas colombianas con escasez de datos todavía mayor.

Y ahí es donde una decisión de diseño que ya estaba en nuestra propuesta original (Sección 4) resulta más relevante de lo que parecía al principio: elegimos un SLM y no un LLM grande en parte porque un modelo de 1 a 3 mil millones de parámetros puede correr en hardware modesto, incluso sin depender de una conexión a internet constante. En el contexto colombiano eso no es un detalle menor: aunque la conectividad rural pasó de 32.2 % en 2022 a 56.9 % en 2025 [32], eso todavía deja a más de cuatro de cada diez hogares rurales sin internet, y esas mismas zonas rurales y apartadas Vaupés, Chocó, La Guajira, Amazonas son, no por coincidencia, donde se concentra buena parte de la diversidad lingüística indígena del país. Un traductor que dependa de una API en la nube, como Google Translate o ChatGPT, no es una opción utilizable ahí. Un modelo pequeño que corra localmente sí lo es.

2.4. ¿Por qué importa y a quién afecta?

Las personas afectadas son, principalmente: hablantes de español con dominio limitado del inglés (o viceversa) que dependen de traducción automática o interpretación informal en contextos críticos de salud, justicia o servicios esenciales; profesionales que deben actuar a partir de una traducción que puede no reflejar lo que la persona quiso comunicar; e intérpretes informales (con frecuencia familiares o menores de edad) que terminan asumiendo una carga y un riesgo que no les corresponde [2]. El caso de Willie Ramírez [1] ilustra el extremo de las consecuencias posibles, pero no es el único tipo de evidencia que hay. La literatura sobre traducción de contenido generado por usuarios muestra que las expresiones dialectales y de jerga son una fuente recurrente de errores "críticos" (esos que invierten o distorsionan

por completo el significado original) incluso fuera de contextos médicos: un estudio sobre traducción de tuits en español documentó que la expresión argentina "ni en pedo" ("de ninguna manera") fue traducida de forma literal y sin sentido [6], y un estudio análogo sobre tuits en árabe relacionados con salud mental encontró que las expresiones dialectales explicaban una porción sustancial de los errores críticos analizados [5].

2.5. Evidencia de la magnitud del problema

Según cifras del censo de Estados Unidos, en 2020 había cerca de 40.5 millones de hablantes de español en el país [3], y estudios sobre dominio limitado del inglés (*Limited English Proficiency*, LEP) estiman que una fracción sustancial de esa población (entre el 39 % y el 41 %, según la fuente y el año) reporta un dominio del inglés menor a "muy bueno" [4]. Esta magnitud poblacional, sumada a la evidencia cualitativa de los casos documentados, sustenta que el problema es real y no un supuesto sin evidencia. Sin embargo, el equipo reconoce una limitación: la frecuencia exacta de errores de traducción dialectal en interacciones cotidianas, fuera de los casos extremos reportados, no ha sido medida directamente todavía, y es justamente parte de lo que este proyecto busca aproximar de forma indirecta a través de sus experimentos de evaluación.

2.6. Por qué no es un problema trivial

Resolver este problema no equivale a conectar una interfaz de usuario a una API de traducción existente. Requiere, como mínimo, tomar decisiones no triviales en al menos cuatro frentes. De datos, porque no existen corpus paralelos amplios de jerga y dialecto del español para *fine-tuning*, así que hay que diseñar un proceso de generación sintética a partir de semillas curadas y validar su calidad. De modelado, porque ajustar un modelo pequeño para una tarea con tanta variabilidad léxica y cultural implica decisiones de arquitectura, tamaño de modelo y técnica de *fine-tuning* (por ejemplo, LoRA) que afectan directamente el equilibrio entre eficiencia y calidad. Experimentales, porque comparar generadores de datos y técnicas de fusión de modelos exige un diseño controlado que aísle bien las variables relevantes. Y de arquitectura de sistema, porque el objetivo no es un script de entrenamiento local sino un servicio desplegable en la nube, con una API que pueda integrarse a otros sistemas (por ejemplo, un servicio de atención al cliente o una plataforma de salud) y que sea observable y escalable.

2.7. Preguntas de investigación

Este proyecto se enmarca principalmente como un problema científico y técnico, motivado por un problema social y organizacional real. Las preguntas de investigación que lo guían son:

- **PI1.** ¿Cómo afecta la elección del LLM utilizado para generar datos sintéticos de entrenamiento a la calidad de la traducción de un SLM ajustado para jerga y dialectos del español?
- **PI2.** ¿Puede la fusión de pesos (simple o guiada por destilación de conocimiento multi-maestro) consolidar en un único modelo el conocimiento de varios SLMs entrenados con datos sintéticos de distintas fuentes, igualando o superando el desempeño del mejor modelo individual?
- **PI3.** ¿Es posible obtener, mediante este enfoque, un modelo pequeño y eficiente cuya calidad de traducción de expresiones dialectales sea competitiva frente a sistemas de traducción de propósito general?

3. Estado del arte

3.1. Traducción automática y su fragilidad ante la jerga y los dialectos

Que a la traducción automática le cueste el texto que no es estándar no es nada nuevo: llevamos más de una década documentándolo. Michel y Neubig [7] armaron un banco de pruebas específico para

medir cómo les iba a los sistemas de MT con texto ruidoso sacado de Reddit, y encontraron que basta con que la entrada no sea "limpia" para que hasta los sistemas más avanzados del momento terminaran produciendo traducciones desastrosas. Pero su banco de pruebas mete en la misma bolsa todo tipo de ruido, sin distinguir de dónde viene. El trabajo más reciente sobre contenido generado por usuarios sí aísla específicamente el componente dialectal del error, y confirma el mismo patrón para español y árabe: buena parte de los errores de traducción "críticos" (esos que terminan invirtiendo el sentido del mensaje original) vienen justamente de expresiones dialectales [5,6]. Esta evidencia motiva directamente el problema que planteamos en la Sección 2, pero ojo —ninguno de estos trabajos propone una solución de *fine-tuning*, se quedan en caracterizar el error.

3.2. Traducción dialectal mediante ajuste fino de modelos de lenguaje

La propuesta que más se parece a la nuestra viene de trabajo reciente sobre dialectos de otros idiomas. Yakhni y Chehab [8] ajustaron el modelo Aya23-8B para traducir el dialecto libanés, probando instrucciones básicas, contrastivas y basadas en reglas gramaticales, y generaron parte de sus datos de entrenamiento con un LLM (Claude 3.5 Sonnet) a partir de un libro de gramática. Sus hallazgos nos sirven bastante: primero, que ajustar el modelo con datos culturalmente auténticos, aunque el conjunto sea chiquito, le gana consistentemente a entrenar con conjuntos más grandes pero traducidos de forma no nativa; y segundo, que entrenar por currículum (en fases sucesivas sobre distintos tipos de instrucción) no mejora nada, algo que ellos mismos atribuyen a olvido catastrófico y que sugieren podría resolverse entrenando modelos por separado y combinándolos después, justo la idea de fusión de modelos que nosotros queremos investigar de forma sistemática. Algo parecido pasa con el aromanian, donde Jerpelea et al. [9] muestran que un corpus paralelo diverso de unos 79 mil ejemplos ya alcanza para ajustar modelos preentrenados con buenos resultados, y con el šariš, donde Ondrejová y Šuppa [10] evalúan qué tan bien traducen los LLMs este dialecto eslovaco de bajo recurso. En el terreno de la jerga (que no es lo mismo que el dialecto regional, aunque están relacionados) Liang et al. [11] arman un banco de pruebas de traducción de jerga inglés-chino con más de 40 mil pares, donde su modelo especializado (SlangOWL) le gana a las líneas base sin ajuste fino.

Para el español, en cambio, lo que hay se queda en el reconocimiento dialectal, no en la traducción. Martínez et al. [12] publican un set de preguntas de opción múltiple para ver si los LLMs distinguen siete macro-variantes del español (andina, antillana, chilena, caribeña continental, mexicana/centroamericana, peninsular, rioplatense), pero sin ajustar ningún modelo para esa tarea. Zander-Marzano et al. [13], en cambio, sí lo hacen: ajustan cuatro LLMs pequeños (LLaMA, Gemma, Mistral, DeepSeek) para clasificar el registro formal/informal en cinco países hispano hablantes, y encuentran que la interferencia de jerga y morfología compartida entre países vecinos es una de las dos fuentes principales de error de clasificación, algo que refuerza justo lo que veníamos diciendo: la jerga y el dialecto complican hasta tareas más simples que la traducción. Pero ninguno de los dos entrena un modelo para traducir lo que detecta.

3.3. Generación de datos sintéticos y comparación de generadores

Como no existen corpus paralelos amplios para dialectos y jergas de bajo recurso, generar datos sintéticos con LLMs se ha vuelto la estrategia de cabecera, tanto para clasificación como para traducción de bajo recurso. Lo que no es tan común es tratar la elección del LLM generador como una variable experimental controlada, o sea, preguntarse en serio si importa cuál LLM usas para generar los datos. Poliakov y Shvai [16] compararon cuatro LLMs generadores (o4-mini, GPT-4.1, GPT-5, o3) para clasificar falacias lógicas, y encontraron diferencias de hasta 14 puntos porcentuales de F1 según cuál generador usaran, aunque hablamos de una tarea de clasificación cerrada, con solo cuatro generadores en juego. Allam et al. [17] usan solo dos generadores, pero se acercan más a lo que nosotros necesitamos porque es generación de texto libre: producen 80 mil pares de preguntas y respuestas médicas sintéticas con dos generadores distintos (GPT-4o y Gemini 2.5 Pro), y encuentran que los datos de GPT-4o dan consistentemente mejores puntajes F1 y menos alucinaciones. Estos dos trabajos son la evidencia más directa de que sí importa quién genera los datos sintéticos, y son los que nos motivan a plantear la Pregunta de

Investigación 1; el problema es que ninguno mide el efecto sobre una tarea de traducción. En una línea relacionada, Schaffelder y Gatt [18] muestran que entrenar con datos sintéticos de fuentes diversas, en vez de una sola, ayuda a evitar el colapso de distribución y mantiene la diversidad del texto generado, lo cual apoya indirectamente la idea de que combinar varias fuentes (ya sea mezclando el *dataset* o fusionando modelos, como planteamos aquí) puede ser mejor que depender de un solo generador.

3.4. Fusión de modelos (*model merging*)

Fusionar los pesos de modelos ajustados por separado se ha vuelto una alternativa liviana al entrenamiento conjunto. Las técnicas más básicas incluyen el promedio simple de pesos o *model soup* [19], la aritmética de tareas (*task arithmetic*), que representa cada ajuste fino como un vector de diferencia respecto al modelo base [20], y refinamientos que atenúan la interferencia entre modelos como TIES-Merging [21] y DARE [22]. La herramienta `mergekit` [23] junta estas técnicas y varias más en un formato accesible mediante archivos de configuración.

Aplicar todo esto a traducción automática, es poco común y la evidencia que hay es mixta. Tao et al. [25] muestran que fusionar modelos ajustados sobre distintos *datasets* funciona razonablemente bien como sustituto de entrenar sobre la mezcla completa de esos datos (con una correlación de Pearson de 0.78 en tareas de visión y 0.57 en tareas de lenguaje), pero sus experimentos son de clasificación y comprensión de lenguaje, no de traducción. En el terreno específicamente multilingüe, Gain et al. [24] fusionan modelos de traducción ajustados de forma independiente sobre distintos pares de idiomas, y encuentran una degradación de desempeño sustancial y desigual, sobre todo cuando el idioma de destino cambia entre los modelos fusionados, un resultado que nos sirve como advertencia, aunque su escenario (idiomas distintos) es diferente al nuestro (mismo par de idiomas, pero distinta fuente de datos sintéticos). A diferencia de esa fusión simple por promedio de pesos, el trabajo más reciente sobre fusión guiada por destilación de conocimiento multi-maestro [26] plantea la fusión como un problema de destilación, minimizando la diferencia entre lo que predice el modelo fusionado y lo que predice cada modelo especializado, pero solo lo evalúan en tareas de clasificación, y ellos mismos dejan la extensión a tareas generativas como trabajo futuro.

3.5. Síntesis y vacío identificado

La Tabla 1 resume el vacío que encontramos después de toda esta revisión: cada pieza de nuestra propuesta existe por separado en la literatura, pero la combinación específica que planteamos (traducir, no solo reconocer, jerga y dialecto del español, comparando de forma controlada los generadores sintéticos, y consolidando todo mediante fusión de modelos) no la encontramos en ninguna fuente.

Componente	**Existe para...**	**Vacío identificado**
Traducción de jerga/dialecto vía *fine-tuning*	Libanés [8], aromanian [9], šariš [10], jerga en-zh [11]	No encontrado para español (solo hay reconocimiento [12, 13])
Comparación de LLMs como generadores sintéticos	Clasificación [16], diálogo médico [17]	No aplicado a traducción
Fusión de modelos entrenados con datos de distintas fuentes	Clasificación/QA [25]	No aplicado a traducción; fusión multilingüe estándar es frágil [24]

Tabla 1: Síntesis del vacío identificado en la literatura revisada.

4. Propuesta general de solución

4.1. Visión general

Lo que proponemos es construir y evaluar un *pipeline* completo de cuatro etapas. Primero, curar un banco de semillas con expresiones dialectales y de jerga junto con su traducción correcta. Segundo, generar datos sintéticos de entrenamiento usando tres LLMs generadores distintos, cada uno trabajando por su cuenta sobre el mismo banco de semillas. Tercero, ajustar un SLM, en el rango de 1 a 3 mil millones de parámetros (estamos considerando Llama 3.2 3B, Qwen2.5 3B y Gemma 2 2B como candidatos), mediante LoRA, sobre cada uno de los tres conjuntos sintéticos que resulten. Y cuarto, consolidar los tres modelos especializados fusionando sus pesos, comparando una fusión simple (TIES/DARE vía `mergekit`) contra una fusión guiada por destilación multi-maestro. Todo esto lo pensamos como un servicio desplegable en la nube, expuesto mediante una API, no como un simple script de entrenamiento local, con observabilidad desde el diseño inicial (métricas de calidad por dialecto, latencia, tasa de retroalimentación).

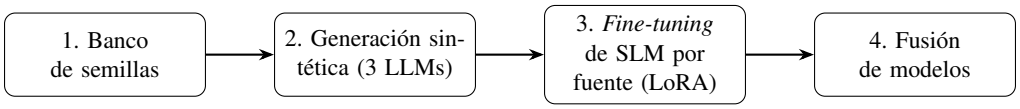


Figura 1: Pipeline conceptual de cuatro etapas de la propuesta: curación de semillas, generación sintética multi-LLM, ajuste fino por fuente, y fusión de modelos.

Como referencia para calibrar qué tan ambicioso puede ser el diseño experimental de las siguientes fases, declaramos el presupuesto de tiempo y cómputo que tenemos disponible en esta etapa como equipo: cada uno de los tres integrantes dedica entre 5 y 8 horas semanales al proyecto (entre 15 y 24 horas semanales combinadas), de forma compaginada con el resto de la carga académica del curso, sin dedicación exclusiva de nadie. En cuanto a cómputo, ninguna de las máquinas del equipo tiene GPU utilizable para ajustar un SLM de 3 mil millones de parámetros ni siquiera con LoRA (una tiene GPU integrada sin CUDA real, otra solo 2GB de VRAM), así que todo el entrenamiento, la fusión de modelos y la evaluación a gran escala se ejecutan sobre la capa gratuita de Google Colab (GPU T4/L4 según disponibilidad), sin costo de cómputo adicional para el proyecto.

4.2. Investigación y producto: dos preguntas distintas

Vale la pena separar dos preguntas que a primera vista parecen la misma, pero no lo son. La primera es por qué este proyecto vale la pena como trabajo de investigación de un semestre: porque nadie ha comparado de forma controlada distintos generadores de datos sintéticos ni ha probado la fusión de modelos para esta tarea específica, como se estableció en la Sección 3. La segunda es por qué alguien usaría lo que construyamos en vez de Google Translate, DeepL, o simplemente pedirle a ChatGPT que traduzca, y esa es una pregunta de producto, no de método, con una respuesta distinta.

La Tabla 2 resume esa segunda respuesta. Ninguna de esas diferencias depende de que nuestro método resulte ser el mejor posible; son ventajas que vienen directamente de haber elegido un SLM desplegable localmente en vez de envolver la API de un LLM grande.

Ninguna de las dos preguntas reemplaza a la otra. Si solo tuviéramos la respuesta de producto sin la de investigación, este sería un proyecto de conectar una interfaz a una API existente, justo lo que la rúbrica del curso marca como trivial. Y si solo tuviéramos la pregunta de investigación sin validar que el resultado sirve para algo real, nunca sabríamos si nuestros hallazgos importan fuera de un paper.

4.3. Decisiones técnicas y científicas a investigar

Hay varias decisiones que todavía no están resueltas y que van a ser parte del trabajo de investigación en sí: qué SLM base usar y con qué técnica de ajuste fino (LoRA vs. ajuste fino completo); cómo diseñar

Atributo	Google Translate / DeepL / ChatGPT	Nuestra propuesta
Conexión a internet	Obligatoria	No obligatoria; puede correr en el dispositivo
Datos sensibles (salud, legal)	Salen a la nube del proveedor	Pueden quedarse dentro de la infraestructura del cliente
Personalización	Caja negra, no ajustable	Se puede re-entrenar con la mezcla dialectal propia de cada cliente
Costo a gran volumen	Pago por llamada a un modelo grande	Modelo pequeño, barato de correr a escala
Auditabilidad	No se sabe con qué se entrenó	Se documenta el origen de cada dato de entrenamiento

Tabla 2: Diferencias de producto frente a las alternativas existentes, independientes del resultado del método de investigación.

el proceso de "derivación" con el que cada generador expande las semillas en variantes de contexto, registro y tono; qué criterios usar para validar y filtrar los datos sintéticos (automáticos y con muestreo humano); qué técnica de fusión conviene más, sabiendo que fusionar adaptadores LoRA es más frágil que fusionar modelos completos; y cómo diseñar la arquitectura de despliegue (multi-tenant, observable, escalable) para que esto no se quede en un simple envoltorio sobre una API que ya existe. Un riesgo real en esta etapa es no tener suficiente acceso a GPU para hacer *fine-tuning* (LoRA) de varios SLMs en paralelo; si eso pasa, vamos a mitigarlo priorizando un solo SLM base y reduciendo cuántos generadores sintéticos comparamos en la implementación inicial, o usando créditos de cómputo gratuitos o baratos (por ejemplo, Google Colab o créditos académicos).

4.4. Alternativa no basada en IA

Como punto de comparación, y para justificar por qué de verdad se necesita un componente inteligente, consideramos como alternativa un glosario o diccionario curado de equivalencias por dialecto, revisado por hablantes nativos. Es una opción más barata y totalmente explicable, pero no generaliza a construcciones nuevas ni a combinaciones de jerga y contexto que no se hayan previsto, que es justo la limitación que este proyecto quiere superar.

4.5. Atributos de calidad e hipótesis a evaluar

Más adelante en el proyecto vamos a evaluar, como mínimo: (i) la calidad de traducción, con métricas automáticas (BLEU, chrF) y evaluación humana de qué tanto se conservan los matices, comparando cada modelo individual, el modelo fusionado, y sistemas de referencia de propósito general; (ii) la eficiencia, comparando tamaño y latencia del SLM resultante frente a modelos más grandes; (iii) el efecto del generador sintético sobre la calidad del modelo (PI1); (iv) qué tan efectiva es la fusión frente al mejor modelo individual y frente a un modelo entrenado sobre la mezcla completa de los tres conjuntos de datos (PI2 y PI3); y (v) la portabilidad del modelo resultante, es decir, si puede correr en hardware modesto y sin conexión constante a internet, algo relevante en zonas rurales con conectividad limitada, como se discutió en la Sección 2.

La evaluación humana de retención de matices la vamos a hacer con un mínimo de tres hablantes nativos por variante dialectal cubierta en el banco de semillas, reclutados por plataformas de anotación remunerada (por ejemplo, Prolific) o por contactos directos del equipo, con un cuestionario de filtro que confirme su origen dialectal. Vamos a medir el acuerdo entre evaluadores con el coeficiente kappa de Cohen (si son dos anotadores) o de Fleiss (si son tres o más), y vamos a reportar ese nivel de acuerdo junto con los resultados de calidad.

5. Contribuciones

En términos generales, este proyecto espera dejar cuatro cosas concretas:

1. Evidencia de que sí es posible ajustar un modelo de lenguaje pequeño (SLM) para traducir jergas y dialectos del español, y no solo para reconocerlos o clasificarlos, que es hasta donde ha llegado la mayoría de trabajos que revisamos.

2. Una comparación controlada de cómo cambia la calidad de un modelo de traducción según qué LLM se use para generar sus datos de entrenamiento sintéticos. Este tipo de comparación ya se ha hecho para otras tareas, como clasificación de falacias lógicas o generación de diálogo médico [16, 17], pero nosotros la aplicamos por primera vez a la traducción de dialectos.

3. Evidencia sobre si es posible fusionar varios modelos de traducción (cada uno entrenado con datos sintéticos de una fuente distinta) en uno solo, ya sea con una fusión simple o guiada por destilación de conocimiento entre varios modelos maestros. Es un escenario que el trabajo previo en fusión de modelos generativos todavía no ha probado [26].

4. Un *pipeline* completo y documentado (generación de datos, validación y evaluación) que cualquier otro equipo pueda reutilizar si quiere extender este trabajo a otras variantes del español que nosotros no alcancemos a cubrir, o incluso, como base metodológica, a lenguas indígenas colombianas con escasez de datos todavía mayor (Sección 2) —algo fuera del alcance técnico de este proyecto, pero que el diseño del *pipeline* no descarta.

5. Más allá del aporte científico, un prototipo que, de resultar viable, ofrece ventajas de producto frente a Google Translate, DeepL o un LLM de propósito general: no depende de conexión a internet constante, no requiere que datos sensibles salgan a la nube de un tercero, es personalizable por cliente y es auditable en cuanto al origen de sus datos de entrenamiento (Sección 4, Tabla 2).

6. Arquitectura (Fase 2)

La Fase 1 dejó la arquitectura como una visión de cuatro etapas sin desarrollar (Figura 1). Esta sección describe qué tanto de esa visión ya es real a la fecha de esta entrega, empezando por el nivel de negocio y bajando hasta datos, aplicación y tecnología, más la gobernanza del proyecto y la hoja de ruta hacia el cierre de esta fase. Un punto se repite en los tres frentes técnicos: por decisión de alcance, el prototipo de la Fase 2 cubrió un solo generador sintético (de los tres considerados) y dejó la fusión de modelos para la fase final, para construir y probar primero el *pipeline* de punta a punta con una sola fuente de datos. Esa fase final ya se ejecutó en lo cuantitativo (tres generadores, cuatro adaptadores y cuatro fusiones; Sección 8); el servicio desplegado ya sirve el adaptador fusionado (PEFT, promedio normalizado de los tres generadores) en vez del adaptador individual del Generador 1 con el que se verificó originalmente el despliegue.

6.1. Arquitectura empresarial

Antes de bajar a datos, aplicación y tecnología, vale la pena dejar explícito el nivel de negocio del que se derivan esas tres capas —la Fase 1 ya lo desarrolló (Sección 2), pero no estaba etiquetado ahí como una capa de arquitectura en sí misma.

Interesados (*stakeholders*). Tres grupos, ya identificados en la Sección 2: hablantes de español con dominio limitado del inglés (o viceversa) que dependen de traducción automática en contextos críticos de salud, justicia o servicios esenciales; profesionales que actúan a partir de esa traducción sin poder verificarla; e intérpretes informales (con frecuencia familiares o menores de edad) que terminan asumiendo un riesgo que no les corresponde. A estos se suma un cuarto grupo, propio de esta fase: la **organización cliente** que adoptaría el servicio (por ejemplo, una empresa de atención al cliente o un proveedor de

salud que atiende población hispanohablante diversa), que es quien efectivamente opera la API y asume su gobernanza de datos (Sección 6.6).

Capacidad de negocio habilitada. Ninguna de las alternativas existentes (Sección 4) le permite a una organización así incorporar traducción especializada en jerga/dialecto sin exponer datos sensibles a un tercero o depender de conexión a internet constante. El servicio de esta fase habilita exactamente esa capacidad —traducción dialectal auditable y desplegable dentro de la propia infraestructura del cliente— resumida en la Tabla 2 de la Sección 4, que no se repite aquí para no duplicar contenido.

Alineación con el modelo de negocio. El diferenciador (portabilidad, no exposición de datos a la nube de un tercero, personalización por cliente, auditabilidad) es una consecuencia directa de una decisión ya tomada en la Fase 1 —usar un SLM en vez de envolver la API de un LLM grande—, no algo añadido en esta fase; lo que esta fase agrega es la evidencia de que esa decisión de negocio efectivamente se sostiene en un sistema real (Sección 7).

6.2. Arquitectura de datos

El *pipeline* de datos tiene cinco pasos, cada uno con su propio script y su propia salida versionada: (1) un **banco de semillas** de expresiones dialectales y de jerga, cada una con dialecto, registro (formal/informal/jerga) y traducción de referencia como campos separados a propósito —dialecto es una dimensión geográfica y registro una de formalidad, y mezclarlas habría sesgado el análisis posterior—; (2) una plantilla de **derivación** que le pide a un LLM generador expandir cada semilla en 5-8 variantes de contexto/registro/tono, sin inventar ni mezclar el dialecto original con otro, y en formato JSON estructurado igual sin importar cuál LLM la reciba; (3) **generación sintética** automatizada, con reintentos ante límites de tasa y guardado de la salida cruda antes de cualquier procesamiento; (4) **validación automática** mediante reglas de filtrado (longitud, casi-duplicados de la semilla original, idioma inesperado, duplicados exactos); y (5) **splits y *data card***, divididos 80/10/10 *por semilla completa* (no por variante) y estratificados por dialecto, con verificación automática de que ninguna semilla queda repartida entre splits.

El banco de semillas cubre 100 semillas en 5 macro-variantes del español (caribeña, andina, rioplatense, mexicana y chilena). Las 100 pasaron por el resto del *pipeline* con cada uno de los tres LLMs generadores (Groq, modelo `openai/gpt-oss-20b`; Cohere, `command-r-08-2024`; y Google, `gemini-3.5-flash-lite`), con la misma plantilla y el mismo filtro, y se reparten en 81, 10 y 9 semillas para entrenamiento, validación y prueba con un reparto fijo, idéntico para los tres generadores. Generaron 589, 632 y 625 variantes; el filtro descartó 2 (0.3 %), 0 y 0, muy por debajo del límite de 20 % fijado como criterio de calidad, lo que también significa que el filtro casi no distingue entre generadores. El Generador 3 pasó de `gemini-3.6-flash` a su versión *lite* porque el nivel gratuito del primero permite solo 20 solicitudes por día. Somos explícitos sobre un hueco: las reglas de filtrado no detectan mezcla de dialectos, y una búsqueda simple del marcador rioplatense "che" encontró 6 variantes del Generador 1 en semillas caribeñas, andinas y chilenas, y ninguna en los otros dos; es un indicio con una heurística de un solo marcador, no una medición de la contaminación, y queda documentado como limitación conocida, no oculto.

6.3. Arquitectura de aplicación

El prototipo (Sección 7) es un servicio de **API REST** (FastAPI), no un script de entrenamiento local: expone traducción, salud, métricas y retroalimentación, con el modelo cargado una sola vez al iniciar el proceso. Sobre esa base se construyeron, en orden: contenerización (Docker/Podman), seguridad básica (límite de tasa por IP, validación de entrada, garantía verificable de que el servicio nunca persiste el texto de ninguna solicitud ni traducción), observabilidad (latencia y tasa de retroalimentación positiva desglosadas por dialecto, sin exponer nunca datos de una solicitud individual) y pruebas de carga.

6.4. Arquitectura de tecnología

Modelo de lenguaje pequeño. De los tres candidatos declarados en la Fase 1 (Llama 3.2 3B, Qwen2.5 3B, Gemma 2 2B), se usó **Qwen2.5-3B-Instruct** en vez de Llama 3.2 3B, nuestro candidato principal original, porque el repositorio de Llama en HuggingFace requiere aprobación manual de Meta y esa solicitud no se había resuelto al momento de necesitar el modelo —no es un cambio de rumbo respecto a lo planteado en la Fase 1, es que el orden de preferencia original quedó bloqueado por un trámite externo, no por un criterio técnico nuestro.

Precisión y cómputo. El modelo se carga en `bfloat16` sin cuantizar (6-6.5GB en memoria) porque el cuello de botella real nunca fue la memoria disponible, sino la ausencia de una GPU utilizable en las máquinas del equipo —un solo paso de entrenamiento LoRA llegó a tardar 80-95 minutos en CPU, completando apenas el 5 % de una prueba de 150 pasos en 10 horas—. Por eso todo el cómputo de ajuste fino, fusión futura y evaluación masiva corre en Google Colab (GPU T4/L4 gratuita), nunca en máquina local, política que dejamos por escrito para el resto del proyecto.

Técnica de ajuste fino: LoRA. Elegida sobre ajuste fino completo por la misma razón que motivó elegir un SLM: mantener bajos el costo de entrenamiento y el tamaño del artefacto resultante, alineado con el objetivo de portabilidad. La Tabla 3 resume la configuración usada, con la justificación de cada valor.

Hiperparámetro	Valor	Por qué
`r` (rango)	8	Capacidad modesta apropiada para un modelo $\sim$3B con un dataset chico; mantiene el adaptador liviano.
`lora_alpha`	16	Heurística estándar $\alpha = 2r$, válida si r cambia al escalar el entrenamiento.
`lora_dropout`	0.05	Regularización barata contra sobreajuste —relevante porque en efecto se observó sobreajuste con esta configuración (ver Sección 8).
`bias`	`"none"`	Valor por defecto estándar de LoRA para modelos de lenguaje causales.
`target_modules`	Q/K/V/O	Proyecciones de atención: mayor impacto al adaptar el modelo a una tarea nueva, sin tocar las capas MLP, para un adaptador más chico y un entrenamiento más rápido.

Tabla 3: Configuración de LoRA usada en los cuatro adaptadores (Generadores 1, 2, 3 y mezcla; idéntica en todos, verificado comparando los archivos de configuración), con la justificación de cada hiperparámetro.

Cada ejemplo de entrenamiento usa el mismo formato de instrucción (sistema/usuario/asistente vía la plantilla de chat nativa del tokenizador) tanto en entrenamiento como en inferencia, con la pérdida enmascarada sobre los tokens del *prompt* para entrenar solo sobre la respuesta esperada —verificado decodificando tokens de un ejemplo real y confirmando que coincide carácter por carácter con el texto original.

6.5. Escenarios de atributos de calidad

Los atributos de calidad declarados en la Fase 1 (Sección 4) se formalizan aquí como escenarios concretos (estímulo/entorno/respuesta/medida), cada uno ya verificado con evidencia real, no solo declarado (Tabla 4).

6.6. Gobernanza del proyecto

El equipo no tiene una jerarquía formal, pero cada persona es dueña de una porción del *pipeline* (datos y evaluación; infraestructura, generación y entrenamiento; estructura del repositorio, filtros y ob-

Atributo	Estímulo	Entorno	Respuesta esperada	Medida (verificada)
Latencia	Un cliente envía una solicitud de traducción	Operación normal, modelo ya cargado	El servicio genera y devuelve la traducción	50-80s por solicitud sin GPU (79.4s/49.8s medidos); con GPU real (ZeroGPU, servicio público): 2.5 s medidos para una solicitud individual, ver Sección 7.
Seguridad	Más de 10 solicitudes en 60s desde el mismo cliente	Operación normal o ráfaga de abuso	El servicio rechaza el exceso, no lo procesa	Confirmado con carga concurrente real: exactamente 10 pasan por ventana de 60s, el resto responde `429` con `Retry-After`, 0 excepciones no controladas.
Privacidad / auditabilidad	Cualquier solicitud de traducción, exitosa o rechazada	Operación normal	El servicio nunca persiste ni expone el texto de la solicitud ni de la traducción	Verificado con prueba automatizada: se envía un texto marcador y se confirma su ausencia en `/metricas` y en cualquier log.
Disponibilidad	Ocurre una excepción no prevista durante la generación	Fallo interno (ej. del modelo)	El servicio responde con un error genérico, sigue operando	Verificado simulando el fallo: responde `500` con mensaje entendible, nunca expone el traceback interno.

Tabla 4: Escenarios de atributos de calidad del prototipo, con la medida de respuesta verificada en esta fase.

servabilidad) y documenta sus decisiones en un registro cronológico compartido —las decisiones que afectan el alcance de una fase completa, el modelo base, o la entrega ante el evaluador del curso, sí requieren acuerdo explícito de los tres.

En cuanto a **gobernanza de datos**: todo el banco de semillas, los datasets sintéticos y los splits quedan versionados con procedencia rastreable hasta la semilla original; en cambio, el servicio en producción *nunca* persiste el texto de ninguna solicitud ni traducción —solo métricas agregadas (Sección 7)—, y los resultados de evaluación humana futura se guardarán con el evaluador identificado de forma anonimizada. En cuanto a **gobernanza de modelos**: cada adaptador LoRA vive en una carpeta con el nombre de su generador, y el criterio para promover un *checkpoint* a "final" es la mejor pérdida de validación, no la última época —precisamente para no promover un modelo sobreajustado solo por haber terminado de entrenar último (ver Sección 8). En cuanto a **gobernanza de proceso**: todo cambio relevante al repositorio requiere una entrada real en el registro de bitácora del proyecto (exigido técnicamente, no solo por convención), y el cómputo pesado sigue siempre la misma política de correr en Colab, nunca en máquina local.

Sobre **riesgos éticos**: el banco de semillas excluye a propósito groserías fuertes explícitas —el foco es jerga cotidiana no ofensiva, coherente con un caso de uso de traducción para negocios o atención al cliente—, y se declaran explícitamente los dialectos *no* cubiertos (español peninsular, variantes caribeñas insulares como Cuba o Puerto Rico) como limitación conocida en vez de dar a entender que el sistema cubre "español" en general: esos dialectos se excluyen porque el equipo no tiene forma de reclutar evaluadores humanos nativos de esas regiones, y cubrir un dialecto que después no se puede validar con hablantes reales sería peor que no cubrirlo.

6.7. Hoja de ruta

La Tabla 5 resume lo ya cerrado en esta fase, lo que falta para cerrarla, y lo que corresponde a la fase final del proyecto.

Estado	Qué
Ya cerrado	Banco de semillas, generación sintética, validación, splits, ajuste fino completo de los tres generadores y de la mezcla con detección de sobreajuste, cuatro fusiones (TIES, DARE+TIES, promedio lineal y destilación multi-maestro), comparación cuantitativa con intervalos de incertidumbre contra la línea base, servicio de API con seguridad, observabilidad y pruebas de carga, prueba de integración de punta a punta.
Cerrando esta fase	Evaluación humana con hablantes nativos reales (infraestructura lista, reclutamiento en curso); pruebas de carga del servicio ya desplegado públicamente, limitadas hasta ahora por la cuota diaria de GPU gratuita (ver Sección 7).
Fase final	Evaluación humana con hablantes nativos para PI1, PI2 y PI3; comparación contra sistemas de traducción dedicados y cuantización para CPU (PI3, hoy parcial); repetir con más semillas de prueba, más de una semilla de entrenamiento y otro modelo base.

Tabla 5: Hoja de ruta del proyecto a la fecha de esta entrega.

7. Evidencia de prototipo

El prototipo construido en esta fase es el servicio de API descrito en la Sección 6, con los endpoints de traducción, salud, métricas y retroalimentación, más seguridad básica (límite de tasa de 10 solicitudes/60 segundos por cliente, validación de entrada) y observabilidad (latencia y tasa de retroalimentación positiva por dialecto) ya implementadas y probadas —no solo con pruebas unitarias con el modelo simulado, sino contra el modelo real cargado en la máquina de desarrollo.

Funciona de punta a punta con el modelo real. Se levantó el servicio con Qwen2.5-3B-Instruct más el adaptador LoRA del Generador 1, sin GPU, y se confirmó con solicitudes reales: `GET /salud` responde en ∼7ms, y `POST /traducir` devuelve una traducción correcta y coherente en 50-80 segundos (79.4s en la primera solicitud, con arranque en frío; 49.8s en la siguiente). Contenerizado con Docker/Podman, el mismo servicio respondió `/salud` en ∼36ms y una traducción coherente en 33.8s. **Las traducciones son correctas: el problema medido es latencia, no funcionalidad**, y se reporta así explícitamente porque 50-80 segundos por solicitud no cumple el objetivo de latencia del proyecto sin GPU real.

Despliegue en la nube: público en Hugging Face Spaces. Se eligió Hugging Face Spaces (SDK Gradio, hardware ZeroGPU —GPU real gratuita asignada solo durante cada generación—) tras descartar Render (*free tier* insuficiente en memoria) y Railway (sin *free tier* permanente en 2026); también se evaluó AWS Academy Learner Lab, descartado porque la sesión se apaga sola a los 40 minutos y sus créditos se comparten con el resto del curso. La primera vez, la creación del Space se bloqueó porque la cuenta tenía menos de los 30 días de antigüedad que la plataforma exige para ZeroGPU gratuito en cuentas personales; el equipo esperó con su propia cuenta en vez de usar la de otro integrante, y el 7 de octubre de 2026 el Space quedó creado y en ejecución. El primer *build* falló al cargar el adaptador (la biblioteca PEFT intenta leerlo directamente a la GPU emulada de ZeroGPU); se corrigió cargándolo en CPU. Con el servicio público, una traducción individual tarda 2.5 s, frente a unos 65 s en CPU local. La prueba de aceptación se repitió desde otro dispositivo y otra red (un celular con datos móviles, usando la interfaz web) con el mismo resultado. Una limitación del despliegue: el Space expone solo traducción y salud; las métricas y la retroalimentación existen únicamente en la API contenerizada.

Integración de punta a punta. Una prueba repetible recorre las seis etapas de la cadena (solicitud, validación y límite de tasa, generación con el modelo ajustado, registro de la métrica, respuesta con identificador, retroalimentación y reflejo en `/metricas`); pasó contra el servicio real con el modelo cargado, y falla indicando la etapa exacta si un eslabón se rompe.

Pruebas de carga. Con el límite de tasa por defecto, una ráfaga de 25 solicitudes concurrentes (en niveles de 5 y 20) confirma el comportamiento diseñado: las primeras 10 en total pasan, el resto recibe

`429 Too Many Requests` —la protección contra abuso funciona bajo carga real, no solo en pruebas unitarias—. Con el límite desactivado, para aislar la capacidad de concurrencia de la capa de API: 0 % de errores en 5, 20 y 50 solicitudes concurrentes, con latencia promedio subiendo de forma moderada (0.71s → 0.94s → 1.30s). Esa prueba usó una traducción simulada. Contra el Space desplegado con el modelo real, un primer intento con cliente anónimo agotó la cuota diaria de GPU gratuita de ZeroGPU tras unas cuatro traducciones, sin medir nada en 20 y 50. Repetida con un token autenticado y ya sobre el adaptador fusionado (Sección 7), sí se pudo medir concurrencia real en los tres niveles, aunque de forma parcial por la misma cuota: de 75 solicitudes intentadas se atendieron 28 (5 de 5, 8 de 20, 15 de 50) antes de agotarla. El servicio corre en paralelo, no se serializa: atender 5, 8 y 15 solicitudes a la vez tomó 4.5, 5.6 y 6.1 segundos de principio a fin —si se serializara, 15 solicitudes a unos 3-4 s cada una tomarían 45-60 s, no 6—, Una réplica independiente al día siguiente, desde otra máquina y otra red pero con el mismo token (la cuota es de la cuenta, no de la IP), atendió 53 de las 75 solicitudes —5 de 5, 15 de 20 y 33 de 50— y confirma el paralelismo con más muestra: 33 solicitudes simultáneas resueltas en 18.7 s de principio a fin, cuando serializadas a la latencia sin carga (5.7 s) habrían tomado unos tres minutos. Esa réplica corrige además una lectura de la primera corrida: la latencia sí empeora de forma clara con la concurrencia (mediana de 5.4 s en el nivel 5, 7.9 s en el 20 y 14.9 s en el 50). La primera corrida sugería lo contrario porque, con la cuota casi agotada, solo alcanzaban a completarse las solicitudes que tomaban turno de GPU temprano, mientras las que habrían esperado más en la cola fallaban antes de entrar al promedio. El límite práctico del servicio gratuito sigue siendo la cuota diaria de GPU, no la capacidad de atender solicitudes a la vez.

Qué cubre este MVP. El servicio desplegado ya sirve el adaptador fusionado (PEFT, promedio normalizado de los tres generadores) en vez del adaptador individual del Generador 1 con el que se verificó el despliegue originalmente, y ya se midió su comportamiento bajo carga (20 y 50 concurrentes) con un token autenticado, aunque de forma parcial por la cuota diaria de GPU (ver más abajo). La evaluación humana formal también queda pendiente —existe la infraestructura para reclutarla y calificarla, pero no evaluadores reales confirmados— y por eso ningún resultado de evaluación humana se reporta en la Sección 8 que no venga de personas reales.

8. Resultados

Los resultados de esta sección vienen de **una sola corrida de cada modelo** sobre un conjunto de prueba común de 174 entradas que proceden de **solo 9 semillas** (las mismas para todos los modelos), con un único modelo base (Qwen2.5-3B-Instruct) y con referencias que en su mayoría son sintéticas. Los intervalos de la Tabla 6 son del 95 % y salen de remuestrear semillas completas, porque las variantes de una misma semilla no son independientes; con tan pocas semillas son anchos, y esa es la incertidumbre real. Por eso se redactan como señales y no como conclusiones.

Qué se comparó. Cuatro adaptadores LoRA con configuración idéntica, entrenados con los datos del Generador 1, del 2, del 3 y de la mezcla de los tres; tres fusiones de los adaptadores de los tres generadores hechas con PEFT (TIES, DARE+TIES y promedio lineal), una fusión guiada por destilación multi-maestro, y las dos técnicas de promedio simple y TIES repetidas con `mergekit` sobre los modelos completos (el modelo base con cada adaptador ya incorporado); todos contra el modelo base sin ajustar. Entre los adaptadores individuales la única variable es el *dataset*.

Ajuste fino. Una primera señal sugiere que ajustar con datos sintéticos ayuda: los tres adaptadores individuales superan al modelo base en BLEU con un intervalo que no incluye el cero (entre +3.7 y +5.2 puntos). En chrF, el del Generador 2 no se distingue del base (+1.4).

PI1: efecto del generador (señal tentativa). Los adaptadores de los Generadores 1 y 3 no se distinguen entre sí (diferencia de +0.1 BLEU y −0.5 chrF, con intervalos amplios). El del Generador 2 (Cohere) queda por debajo de ambos en chrF (G1 − G2: +2.3, intervalo [+0.2, +4.2]; G3 − G2: +2.7, [+0.8, +5.3]) y no se distingue en BLEU. Una hipótesis que no probamos es que Cohere generó más registro formal y menos jerga con el mismo *prompt* (20 % de variantes formales y 19 % de jerga, frente a 12 % y 30 % del Generador 1 y 7 % y 33 % del 3). Dos advertencias: el Generador 3 es un modelo

Modelo	BLEU	chrF
Base sin ajustar	37.0 [34.3, 40.1]	53.9 [51.8, 55.9]
LoRA Generador 1 (Groq)	42.2 [38.7, 46.3]	57.5 [55.5, 59.7]
LoRA Generador 2 (Cohere)	40.7 [38.7, 43.3]	55.3 [53.6, 57.1]
LoRA Generador 3 (Google *lite*)	42.3 [39.0, 46.7]	57.9 [55.7, 61.3]
LoRA sobre la mezcla 1+2+3	42.4 [39.9, 45.5]	57.2 [55.2, 59.8]
Fusión TIES	43.7 [40.9, 47.1]	58.2 [56.4, 60.4]
Fusión DARE+TIES	43.8 [40.5, 47.5]	57.7 [55.6, 60.2]
Promedio con PEFT, pesos 1.0 (suma, error de configuración)	35.3 [32.5, 37.5]	51.5 [49.6, 53.8]
Fusión por destilación multi-maestro	43.9 [40.8, 47.5]	58.3 [56.5, 60.5]
Fusión lineal con `mergekit` (modelos completos)	44.2 [41.1, 48.3]	58.4 [56.3, 61.2]
Fusión TIES con `mergekit` (modelos completos)	44.0 [41.1, 47.6]	58.0 [56.2, 60.2]
Promedio con PEFT, pesos normalizados a 1/3	43.6 [40.8, 47.4]	58.1 [56.2, 60.7]
Promedio exacto con PEFT (concatenación), pesos 1/3	43.5 [40.5, 47.5]	58.1 [56.0, 61.1]

Tabla 6: BLEU y chrF globales (intervalo de confianza de 95 % por *bootstrap* sobre semillas) de cada modelo sobre el conjunto de prueba común (174 entradas, 9 semillas).

lite, más pequeño, así que "qué LLM es" no se separa de "qué tamaño tiene"; y cada generador sale mejor en las referencias que escribió su propio LLM (por ejemplo, el Generador 2 con las referencias del Generador 2), un sesgo de la evaluación que el conjunto de referencias humanas del equipo debería corregir, pero tiene solo 9 entradas y no permite concluir nada.

PI2: fusión de modelos (señal tentativa). Todas las fusiones que funcionan quedan en un mismo rango, de 43.7 a 44.2 BLEU y de 57.7 a 58.4 chrF (TIES, DARE+TIES y destilación con PEFT, y promedio simple y TIES con `mergekit`); las diferencias entre ellas son de 0.3 a 0.6 puntos y sus intervalos incluyen el cero, así que con estos datos el método de fusión importa menos que fusionar bien. Frente al mejor adaptador individual, las fusiones hechas con PEFT lo superan por unos 1.4 puntos de BLEU sin que el margen se distinga del ruido (por ejemplo, TIES − Generador 1: +1.4, [−0.7, +4.1]). Con `mergekit` el margen es algo mayor: el promedio simple supera al adaptador del Generador 1 (+2.0 BLEU, [+0.2, +4.2]) y al del 3 (+2.2, [+0.5, +4.2]), y TIES al del 1 (+1.8, [+0.2, +3.5]), con intervalos que no incluyen el cero; en chrF ninguna de esas diferencias se distingue del ruido (de +0.4 a +0.9). Es una señal tentativa y débil: son decenas de comparaciones sobre 9 semillas, así que un intervalo que apenas excluye el cero puede ser casualidad. Todas las fusiones superan de forma distinguible al adaptador del Generador 2, y en BLEU también al entrenado sobre la mezcla, salvo la destilación (+1.3, [−0.5, +3.0]); por ejemplo, TIES con PEFT − mezcla: +1.3, [+0.3, +2.4]. En resumen, una primera señal sugiere que fusionar los adaptadores es al menos tan bueno como el mejor modelo individual y quizás algo mejor en BLEU, y que, en BLEU, sale mejor que entrenar sobre la mezcla de los datos (la destilación aparte). La fusión guiada por destilación no mejora a TIES (diferencia de −0.0 BLEU y +0.1 chrF) y exigió unos 95 minutos de GPU frente a unos 24 segundos de TIES con PEFT, así que con estos datos su costo extra no se justifica. El resultado más instructivo es el del promedio simple: hecho con PEFT y sus pesos por defecto (que suman en lugar de promediar), queda por debajo del modelo base en chrF (−2.4, [−4.4, −0.2]); hecho con `mergekit` sobre los modelos completos da 44.2 BLEU y 58.4 chrF, la diferencia entre ambos es de +9.2 BLEU ([+6.1, +13.3]). Eran en realidad dos diferencias a la vez: con pesos 1.0, PEFT *suma* las tres actualizaciones en lugar de promediarlas, y promediar por separado las dos matrices de cada adaptador introduce seis términos cruzados que ningún adaptador entrenó. Un experimento posterior las separó. Con los pesos normalizados a 1/3, el promedio con PEFT da 43.6 BLEU y 58.1 chrF, y el promedio exacto de las actualizaciones (PEFT con concatenación de adaptadores, sin términos cruzados) da 43.5 y 58.1, sin diferencia distinguible entre ambos (−0.1 BLEU, [−0.7, +0.6]); los dos superan al promedio sin normalizar por +8.5 BLEU ([+5.6, +12.4]). La causa del mal resultado era, por tanto, la escala (sumar en vez de promediar) y no los términos cruzados, aunque en el espacio de pesos estos tienen más energía que la señal (1.2 veces) y las matrices A de adaptadores

distintos son ortogonales; no investigamos por qué esos términos no dañan. Fue un error de configuración nuestro (usamos los pesos por defecto, 1.0), no una limitación de PEFT ni de la fusión lineal. `mergekit` queda 0.7 BLEU por encima de las dos variantes normalizadas, con un intervalo que apenas excluye el cero ([0.0, +1.4]); no interpretamos esa diferencia.

Sobreajuste y costo. En los cuatro entrenamientos la mejor pérdida de validación llega en la primera época y empeora después mientras la de entrenamiento sigue bajando; el mecanismo de parada temprana conservó siempre la primera época. Entrenar un adaptador individual tomó entre 18 y 20 minutos en una GPU T4, y el de la mezcla, con tres veces más ejemplos, unos 52; fusionar con TIES tomó segundos. Un adaptador pesa unos 15 MB frente a unos 6 GB del modelo base.

PI3: portabilidad y competitividad (señal parcial). Sobre portabilidad hay evidencia directa. Un adaptador pesa 14.8 MB (el 0.24 % del modelo base de 6.17 GB), así que cambiar de especialización no cuesta otros 6 GB. El sistema funciona sin internet una vez descargados los pesos: en un proceso con toda conexión de red bloqueada a nivel de *sockets* y con Hugging Face en modo *offline*, la carga del modelo y tres traducciones produjeron 0 intentos de conexión; no equivale a desconectar físicamente la máquina, y el primer arranque sí necesita internet para bajar los pesos. En una CPU sin GPU el proceso ocupó 6.7 GB de RAM y tardó entre 46 y 65 segundos por traducción: portátil, pero no interactivo sin una GPU (con una T4 son unos 1.9 segundos, y 2.5 en el servicio público). Una prueba preliminar con cuantización dinámica a int8 en CPU (3 frases) redujo el modelo a 4.33 GB y la latencia mediana a 28 s, todavía lejos de lo interactivo, pero no midió el efecto sobre la calidad. En GPU sí lo medimos, sobre las 174 entradas del test común y con `bitsandbytes`: frente a fp16 (43.6 BLEU, 6.17 GB), int8 pierde 0.9 BLEU (IC 95 % $[-1{,}6, -0{,}1]$) y ocupa 3.40 GB, y nf4 pierde 1.6 BLEU ($[-2{,}7, -0{,}6]$) y ocupa apenas 2.01 GB; en chrF solo nf4 tiene una pérdida distinguible ($-1{,}3$, $[-2{,}5, -0{,}3]$). Un resultado contraintuitivo que no investigamos a fondo: int8 resultó la variante más lenta de las tres (4.04 s de latencia mediana, frente a 0.83 s en fp16 y 1.25 s en nf4) pese a ocupar menos memoria, un efecto conocido de `bitsandbytes`. Con estos datos, nf4 es el mejor compromiso entre las dos variantes cuantizadas.

Sobre competitividad, comparamos contra LLMs de propósito general con el mismo *prompt* y las mismas métricas (Tabla 7); Google Translate no se evaluó (no teníamos clave de API y no quisimos incurrir en costos), y DeepL se trata aparte, más abajo. Frente a dos modelos grandes que no generaron datos (gpt-oss-120b y qwen3.8-27b), el mejor modelo pequeño queda *por debajo* de forma distinguible: -3.9 BLEU ($[-7.2, -1.5]$) y -4.5 chrF frente al primero, y -3.4 BLEU ($[-6.8, -0.4]$) y -4.0 chrF frente al segundo. Frente a los tres LLM que sí generaron los datos de entrenamiento, evaluados sin sus propias referencias, el modelo pequeño no se distingue de `command-r` (-1.2 BLEU, $[-4.2, +2.3]$) ni de `gemini-3.5-flash-lite` (+0.6, $[-2.8, +4.0]$) en BLEU, queda algo por debajo en chrF (de -2.0 a -2.9, con el cero en el límite del intervalo) y supera a `gpt-oss-20b` en BLEU (+3.5, [+0.2, +7.0]). Esa segunda comparación está sesgada a favor del modelo pequeño: aprendió justamente de las traducciones de esos LLM. Con las referencias humanas del banco de semillas (solo 9 entradas) los LLM grandes puntúan más alto que el modelo pequeño (por ejemplo, 11.5 BLEU frente a 4.9), una brecha que parece mayor que con referencias sintéticas pero que con tan pocos datos no permite concluir. En resumen: el modelo pequeño ajustado y fusionado recorre más de la mitad de la distancia entre el modelo base y los LLM grandes (7.2 de 11.0 puntos de BLEU y 4.5 de 8.9 de chrF) con una fracción del tamaño, pero con estos datos no los iguala.

Frente a DeepL, un sistema de traducción dedicado (API gratuita, mismo test común, sin *prompt* ni conocimiento del dialecto), el modelo pequeño fusionado no se distingue en BLEU (44.2 frente a 44.4; diferencia +0.0, $[-3.9, +3.9]$) y queda por debajo en chrF (58.4 frente a 61.4; -3.1, $[-6.5, -0.2]$, con el intervalo apenas excluyendo el cero). Con las 9 referencias humanas DeepL puntúa más (16.1 BLEU frente a 4.9), con muy pocos datos para concluir. Los casos concretos son mixtos: DeepL acierta expresiones que el modelo pequeño falla ("estar bien pedo" como "to be really drunk", "hacerse humo" como "to vanish") y falla otras, como "neta", que deja sin traducir, o "coger la caña", que traduce literalmente como "pick up the rod". Es decir, un modelo de 3B ajustado con datos sintéticos queda al nivel de un traductor comercial en BLEU, pero no por encima, y no lo superamos en chrF.

Análisis cualitativo de errores (preliminar). Un solo lector, no hablante nativo, clasificó a mano

Sistema	Parámetros	BLEU	chrF
Qwen2.5-3B base, sin ajustar	3B	37.0	53.9
Modelo pequeño: fusión lineal (`mergekit`)	3B + 15 MB	44.2	58.4
qwen3.8-27b	27B	47.7	62.3
gpt-oss-120b	120B	48.0	62.8

Tabla 7: BLEU y chrF globales del modelo pequeño y de dos LLM grandes que no generaron datos de entrenamiento, sobre el mismo test común (173 o 174 entradas; una traducción de qwen3.8-27b falló).

los 34 casos peor puntuados (20 % del test común, por chrF) de la fusión simple con `mergekit`. Unos 16 (47 %) son errores de significado: el modelo elige el sentido estándar de una palabra polisémica regional ("mosca" como "alerta", "pedo" como "borracho", "mala leche") o la reemplaza por una expresión inglesa de apariencia parecida ("coger la caña" → "get your act together"), o traduce literal una comparación ("más salado que un bacalao"); 8 (24 %) conservan el sentido pero pierden el matiz ("al tiro" como "let's go" en vez de "right away"); y 10 (29 %) son traducciones aceptables que la métrica castiga porque la referencia usa otras palabras. Un chrF bajo no equivale, pues, a un error, otra razón por la que hace falta evaluación humana. Las expresiones aisladas (9 entradas) puntúan 14.3 de chrF frente a 59.7 de las oraciones, en parte porque sus referencias son glosas con alternativas. Cada dialecto tiene una o dos expresiones en el test, así que no se separa el efecto del dialecto del de la expresión; la clasificación es subjetiva y debe validarse con nativos.

Evaluación humana: todavía no disponible. A la fecha de esta entrega no existe ninguna ronda de evaluación humana real que reportar. Se preparó la infraestructura completa (rúbrica de retención de matices en escala 1-5 con ejemplos de calibración, hojas de evaluación ciegas por dialecto, cálculo del acuerdo entre evaluadores con kappa de Fleiss y de Cohen, y el formulario de reclutamiento para un mínimo de 3 hablantes nativos por cada uno de los 5 dialectos), pero el reclutamiento real de evaluadores es una acción humana que no se puede simular ni adelantar sin datos falsos. Se hizo, por separado, un muestreo piloto calificado por el propio equipo para probar el formato de la hoja de evaluación —encontrando, entre otras cosas, el caso de mezcla de dialecto mencionado en la Sección 6—, pero deliberadamente *no* se reporta aquí como resultado de evaluación humana, porque no lo es: reportarlo como tal sería engañoso. Como BLEU y chrF miden coincidencia con una referencia y no retención de matices, todo lo anterior queda pendiente de esa evaluación.

Latencia. Medida en la Sección 7: 50-80 segundos por solicitud con el modelo real sin GPU, y 2.5 s con GPU real (ZeroGPU, servicio público; una sola solicitud, sin carga). La diferencia confirma que el despliegue con GPU era necesario, no opcional; bajo concurrencia, con el servicio público, la mediana sube a 5.4 s con 5 solicitudes simultáneas y a 14.9 s con 50 (Sección 7).

Qué significan estos resultados, y qué no. Dan una primera respuesta tentativa a PI1 y PI2 y una respuesta parcial a PI3: es portable y funciona sin internet, pero con estos datos no iguala a los LLM grandes de propósito general y no se comparó contra sistemas de traducción dedicados. Las limitaciones que condicionan todo lo anterior son: solo 9 semillas de prueba; una sola corrida por modelo, sin medir la variación entre semillas aleatorias de entrenamiento; referencias mayormente sintéticas con el sesgo hacia su propio generador descrito arriba; un único modelo base; y la ausencia de evaluación humana. Hay además riesgos de uso sin resolver: el modelo puede traducir sin advertir jerga vulgar o insultante, los datos generados por LLM pueden reproducir estereotipos regionales, solo se cubren cinco dialectos, y no verificamos formalmente las condiciones de licencia de los datos generados con las capas gratuitas de las tres APIs. En costos, todo el proyecto se hizo con capas gratuitas (0 dólares): entre 18 y 20 minutos de GPU T4 por adaptador individual, unos 52 para la mezcla, entre 13 segundos y unos 20 minutos para fusionar y unos 95 para destilar, sin contar en las fusiones el entrenamiento previo de los tres adaptadores.

9. Trabajo futuro

Lo que queda para cerrar el proyecto: la evaluación humana con hablantes nativos reales (mínimo tres por dialecto), con el cálculo de acuerdo entre evaluadores que ya está implementado; completar PI3, comparando también contra Google Translate (DeepL ya se midió); y confirmar si bajar la duración reservada por llamada de GPU (`@spaces.GPU`) permite medir 20 y 50 concurrentes sin que la cuota diaria se agote a mitad de la prueba (Sección 7).

Para saber si las señales de la Sección 8 se sostienen: más semillas de prueba (hoy 9), más de una semilla de entrenamiento por modelo, otro modelo base (por ejemplo Llama 3.2 3B, si Meta aprueba el acceso) y un generador de tamaño comparable al de los otros dos para separar "qué LLM es" de "qué tamaño tiene".

El servicio público ya sirve el adaptador fusionado (PEFT, promedio normalizado de los tres generadores) en vez del adaptador individual del Generador 1 que exponía hasta antes de esta entrega, y ya se midió su comportamiento bajo 20 y 50 solicitudes concurrentes: corre en paralelo, no se serializa —medido en dos corridas independientes, de dos integrantes y desde dos redes distintas—, aunque la medición completa sigue limitada por la cuota diaria gratuita de GPU (Sección 7).

10. Conclusiones

Este documento presentó un proyecto que busca traducir jerga y dialectos del español y el inglés mediante un modelo de lenguaje pequeño ajustado sobre datos sintéticos generados por varios LLMs, y explorar si esa información se puede consolidar fusionando modelos. La revisión de literatura sustentó el problema con evidencia cuantitativa y cualitativa, e identificó un vacío específico —traducción dialectal mediante *fine-tuning*, comparación de generadores de datos sintéticos, y fusión de modelos entrenados con datos heterogéneos, combinados para el español— que hasta donde pudimos establecer nadie ha cubierto todavía.

La propuesta general se convirtió en arquitectura concreta, gobernanza documentada y un prototipo funcional y desplegado: un servicio de traducción con seguridad, observabilidad y pruebas de carga reales, probado de punta a punta con el modelo ajustado. Con los datos de tres LLMs generadores y cuatro técnicas de fusión ya hay una primera respuesta tentativa a las tres preguntas de investigación (la de PI3, parcial: el modelo pequeño es portable y funciona sin internet, pero no iguala a los LLM grandes): los generadores de Groq y de Google dieron adaptadores equivalentes y el de Cohere uno algo peor en chrF, y fusionar los adaptadores es al menos tan bueno como el mejor modelo individual y quizás algo mejor en BLEU (una señal débil), con una destilación que cuesta mucho más que TIES sin mejorarlo y con un promedio simple que funciona igual de bien con `mergekit` sobre modelos completos (44.2 BLEU) que con PEFT sobre los adaptadores (43.6), siempre que se promedie y no se sume: con los pesos por defecto, que sumaban, el resultado cayó a 35.3, un error de configuración nuestro que el experimento permitió identificar. También fuimos explícitos sobre lo que todavía no funciona o no existe —solo 9 semillas de prueba y una corrida por modelo, referencias mayormente sintéticas, la ausencia de evaluación humana, PI3 evaluada solo contra LLM de propósito general, la latencia sin GPU, la mezcla de dialecto sin detectar en los datos, y las pruebas de carga del servicio desplegado limitadas por la cuota gratuita de GPU— porque documentar honestamente las limitaciones nos parece tan parte del método científico como reportar lo que sí salió bien.

Referencias

[1] Price-Wise, G. (2008). *Language, Culture, And Medical Tragedy: The Case Of Willie Ramirez.* Health Affairs Forefront. https://www.healthaffairs.org/do/10.1377/forefront.20081119.000463/

[2] AMS Resources (2026). *Spanish dialects in U.S. legal and medical interpreting: when matching matters*. https://accessmultilingual.com/resources/spanish-dialects-in-us-legal-and-medical-interpreting

[3] U.S. Census Bureau (2022). *Languages We Speak in the United States*. https://www.census.gov/library/stories/2022/12/languages-we-speak-in-united-states.html

[4] Migration Policy Institute (2023). *Language Diversity and English Proficiency in the United States*. https://www.migrationpolicy.org/article/language-diversity-and-english-proficiency-united-states

[5] Saadany, H., Tantawy, A., Orăsan, C. (2024). *Cyber Risks of Machine Translation Critical Errors: Arabic Mental Health Tweets as a Case Study*. arXiv:2405.11668.

[6] Saadany, H., Orăsan, C., Caro Quintana, R., do Carmo, F., Zilio, L. (2021). *Challenges in Translation of Emotions in Multilingual User-Generated Content: Twitter as a Case Study*. arXiv:2106.10719.

[7] Michel, P., Neubig, G. (2018). *MTNT: A Testbed for Machine Translation of Noisy Text*. arXiv:1809.00388.

[8] Yakhni, S., Chehab, A. (2025). *Fine-Tuning LLMs for Low-Resource Dialect Translation: The Case of Lebanese*. arXiv:2505.00114.

[9] Jerpelea, A., Radoi, A., Nisioi, S. (2025). *Dialectal and Low-Resource Machine Translation for Aromanian*. COLING 2025 / arXiv:2410.17728.

[10] Ondrejová, V., Šuppa, M. (2024). *Can LLMs Handle Low-Resource Dialects? A Case Study on Translation and Common Sense Reasoning in Šariš*. Proceedings of VarDial 2024, ACL Anthology 2024.vardial-1.11.

[11] Liang, J., Meng, F., Wang, Y., Zhou, J. (2025). *SlangDIT: Benchmarking LLMs in Interpretative Slang Translation*. arXiv:2505.14181.

[12] Martínez, A., Mayor-Rocher, S., Pozo Huertas, C., Melero, M., Grandury, M., Reviriego, P. (2025). *Spanish is not just one: A dataset of Spanish dialect recognition for LLMs*. Data in Brief, 63, 112088.

[13] Zander-Marzano, D., Hernández-Orallo, J., Martínez-Plumed, F., et al. (2026). *From formal to informal: how register shift confuses LLMs in Spanish dialect recognition*. International Journal of Data Science and Analytics, 22, 205.

[14] Kawasaki, Y. (2026). *Digital Linguistic Bias in Spanish: Evidence from Lexical Variation in LLMs*. arXiv:2602.09346.

[15] Rest of World (2025). *Latin America is building LatamGPT to rival ChatGPT*. https://restofworld.org/2025/chatgpt-latin-america-alternative-latamgpt/

[16] Poliakov, M., Shvai, N. (2025). *MisSynth: Improving MISSCI Logical Fallacies Classification with Synthetic Data*. arXiv:2510.26345.

[17] Allam, A., et al. (2025). *Scaling Arabic Medical Chatbots Using Synthetic Data: Enhancing Generative AI with Synthetic Patient Records*. arXiv:2509.10108.

[18] Schaffelder, M., Gatt, A. (2026). *Synthetic Eggs in Many Baskets: The Impact of Synthetic Data Diversity on LLM Fine-Tuning*. Findings of the Association for Computational Linguistics: ACL 2026, pp. 7265–7293. arXiv:2511.01490.

[19] Wortsman, M., et al. (2022). *Model Soups: Averaging Weights of Multiple Fine-Tuned Models Improves Accuracy Without Increasing Inference Time*. Proceedings of the 39th International Conference on Machine Learning (ICML), pp. 23965–23998.

[20] Ilharco, G., Ribeiro, M.T., Wortsman, M., Gururangan, S., Schmidt, L., Hajishirzi, H., Farhadi, A. (2022). *Editing Models with Task Arithmetic*. arXiv:2212.04089.

[21] Yadav, P., Tam, D., Choshen, L., Raffel, C., Bansal, M. (2023). *TIES-Merging: Resolving Interference When Merging Models*. Advances in Neural Information Processing Systems (NeurIPS) 36.

[22] Yu, L., Yu, B., Yu, H., Huang, F., Li, Y. (2024). *Language Models are Super Mario: Absorbing Abilities from Homologous Models as a Free Lunch*. Proceedings of the 41st International Conference on Machine Learning (ICML).

[23] Goddard, C., Siriwardhana, S., Ehghaghi, M., Meyers, L., Karpukhin, V., Benedict, B., McQuade, M., Solawetz, J. (2024). *Arcee's MergeKit: A Toolkit for Merging Large Language Models*. Proceedings of EMNLP 2024, Industry Track, pp. 477–485.

[24] Gain, B., Ekbal, A., Singh, T.N. (2026). *One Model to Translate Them All? A Journey to Mount Doom for Multilingual Model Merging*. arXiv:2604.02881.

[25] Tao, Z.S., Vinken, K., Yeh, H.W., Cooper, A., Boix, X. (2025). *Merge to Mix: Mixing Datasets via Model Merging*. arXiv:2505.16066.

[26] Dalili, S.A., Mahdavi, M. (2025). *Model Merging via Multi-Teacher Knowledge Distillation*. arXiv:2512.21288.

[27] Google (2025). *Bringing state-of-the-art Gemini translation capabilities to Google Translate*. Google Blog. https://blog.google/products/search/gemini-capabilities-translation-upgrades/

[28] DeepL Help Center (2026). *DeepL Translator languages*. https://support.deepl.com/hc/en-us/articles/360019925219-DeepL-Translator-languages

[29] Jimenez, I.D., Kern, C. (2026). *Dialect and Gender Bias in YouTube's Spanish Captioning System*. arXiv:2602.24002.

[30] Linglass (2026). *YouTube Auto-Translate Subtitles Not Working? Here's Why*. https://linglass.app/blog/youtube-auto-translate-not-working

[31] El Tiempo (2025). *Lenguas indígenas, un tesoro nacional en peligro*. https://www.eltiempo.com/colombia/otras-ciudades/lenguas-indigenas-un-tesoro-nacional-en-peligro-3417164

[32] Cambio Colombia (2026). *Gobierno Petro conectó 3,5 millones de hogares y amplía el acceso a internet en zonas rurales*, con base en la Encuesta de Calidad de Vida 2025 del DANE. https://cambiocolombia.com/contenido-especial/articulo/2026/5/gobierno-petro-conecto-35-millones-de-hogares-y-amplia-el-acceso-a-internet-en-zonas-rurales/